

CS2308算法与复杂性 书面作业

uu-matter543

2024–2025–2

目录

1 Homework 1	2
1.1 Week 1–20250220	2
1.2 Week 2–20250227	5
2 Homework 2	6
2.1 Week 3–20250306	6
2.2 Week 4–20250313	9
3 Homework 3	12
3.1 Week 5–20250320	12
3.2 Week 6–20250327	15
4 Homework 4	17
4.1 Week 7–20250403	17
4.2 Week 8–20250410	17
5 Homework 5	19
5.1 Week 9–20250417	19
5.2 Week 10–20250424	21
6 Homework 6	23
6.1 Week 12–20250508	23
6.2 Week 13–20250515	24
7 Homework 7	25
7.1 Week 14–20250522	25
7.2 Week 15–20250529	26

1 Homework 1

1.1 Week 1-20250220

Problem 1 一块长方形巧克力由 $m \times n$ 个小块组成，每一步只能拿起 1 块大于 1×1 的巧克力，沿分割线将其分割成 2 块

证明：无论采取何种顺序，将巧克力分为 1×1 所需的步数均相同

Solution 1 对块数进行归纳

- 每掰 1 次，总块数 +1
- 开始为 1 块，最终为 $m \times n$ 块
- 总是需要 $m \times n - 1$ 步

Solution 2 对边数进行归纳

- 每掰 1 次，总边数 +4
 - 将原先的 2 条长边分别分割为 2 条短边
 - 原本不存在的 2 条中间边出现
- 开始为 4 条边，最终为 $4 \times m \times n$ 块
- 总是需要 $m \times n - 1$ 步

Problem 2 数列 $1, 2, 3, 4, 5, 10, 20, \dots$ ，从第 5 项起后面为公比为 2 的等比数列

证明：任意正整数均可表示为数列中不重复的数之和

- $\forall x \leq 10$ ，显然命题成立
- 强归纳法：设 $\forall x \leq a$ 成立，其中 a 是数列中的项
 - 由归纳假设， $a < x < 2a$ 时， $x - a < a$ 可用数列中小于 a 的数表示，则 x 可用 $x - a$ 的序列与 a 之和表示，命题成立
 - 而 $2a$ 也是数列中的项，命题成立
- 由递推关系，命题对 $\forall x \leq 2^n a$ 均成立，因此命题对任意正整数均成立

Problem 3 n 个海盗瓜分 100 个金币。将 n 个海盗编号为 $1, 2, 3, \dots, n$ ，先由 n 号海盗提出分配方案，若通过人数 $\geq \frac{t}{2}$ (t 为当前人数)，则按上述分配方案分配，否则将 n 号海盗丢入大海喂鲨鱼；之后由 $n-1$ 号海盗分配。海盗们最希望获得更多金币；获得金币数相同时，则更希望将同伴丢入大海喂鲨鱼。

求： $n = 2, 20, 200, 2000$ 的最优分配方案（首先保证活命，在能够活命的前提下尽可能分给自己更多金币）

- $n = 1$, 分给自己 100 个金币即可
- $n = 2$, 分给自己 100 个金币即可
- $n = 3$, 分给自己 99 个金币, 1号 1 个金币即可
 - 2号无论如何都会反对3号: 因为将3号丢入大海喂鲨鱼后, 2号既能活命, 又能获得 100 个金币; 所以无需分给2号金币
 - 由2号分配时, 1号将会获得 0 金币; 所以3号分给1号 0 个金币时, 1号会反对3号, 因为获得的金币数相等时, 更倾向于将3号丢入大海喂鲨鱼
 - 3号为了使方案通过, 至少要给1号 1 金币, 故方案如上
- $n = 4$, 分给自己 99 金币, 2号 1 个金币即可
 - 与上述分析同理, 3号至少要获得 100 个金币才会支持4号, 2号只要获得 1 个金币就会支持4号, 1号至少要获得 2 个金币才会支持4号
 - 所以为了得到 2 票, 4号给别人最少金币的方案为让2号支持自己, 故方案如上
- $n = 5$, 分给自己 98 金币, 3号和1号各 1 金币
- $n = 6$, 分给自己 98 金币, 4号和2号各 1 金币
- $\vdots$
- $n = 200$, 分给200号、198号、196号、……、2号各 1 金币
- $n = 201$, 201号可以用 100 个金币分别使199号、197号、195号、……、1号支持自己, 加上自己的 1 票, 共 101 票, 方案能通过
- $n = 202$, 202号可以用 100 个金币分别使200号、198号、196号、……、2号支持自己, 加上自己的 1 票, 共 101 票, 方案能通过
- $n = 203$, 按照 $n = 202$ 的分析, 203号无法用 100 个金币获得 101 个海盗的支持, 无法获得 102 票, 因此一定会被丢入大海喂鲨鱼
- $n = 204$
 - 根据 $n = 203$ 的分析, 203号会无条件支持204号, 否则204号被丢入大海喂鲨鱼后, 203号也会被丢入大海喂鲨鱼
 - 考虑到204号自身和203号的无条件支持, 以及由202号分配时, 有 102 个海盗会获得 0 个金币, 因此可以用 100 个金币获得其中 100 个海盗的支持, 这样总共 102 票, 方案能通过
- $n = 205$
 - 由于204号分配时有通过方案, 所以分给203号和204号 0 金币时, 203号和204号也会反对
 - 所以205号无法用 100 个金币获得 102 个海盗的支持, 无法获得 103 票, 因此一定会被丢入大海喂鲨鱼
- $n = 206$

- 根据 $n = 205$ 的分析, 205号会无条件支持206号, 否则206号被丢入大海喂鲨鱼后, 205号也会被丢入大海喂鲨鱼
- 虽然有206号自身和205号的无条件支持, 但按照 $n = 204$ 的分析, 205号无法用 100 个金币获得 101 个海盗的支持, 无法获得 103 票, 因此一定会被丢入大海喂鲨鱼
- $n = 207$
 - 根据 $n = 206$ 的分析, 205号和206号会无条件支持207号, 否则207号被丢入大海喂鲨鱼后, 206号和205号也会先后被丢入大海喂鲨鱼
 - 虽然有207号自身和205号、206号的无条件支持, 但按照 $n = 204$ 的分析, 207号无法用 100 个金币获得 101 个海盗的支持, 无法获得 104 票, 因此一定会被丢入大海喂鲨鱼
- $n = 208$
 - 根据 $n = 207$ 的分析, 205号、206号、207号会无条件支持208号, 否则208号被丢入大海喂鲨鱼后, 207号、206号、205号也会先后被丢入大海喂鲨鱼
 - 考虑到208号自身和205号、206号、207号的无条件支持, 以及由204号分配时, 有 104 个海盗会获得 0 个金币, 因此可以用 100 个金币获得其中 100 个海盗的支持, 这样总共 104 票, 方案能通过
- ⋮
- 因此最终能存活下来的海盗编号如下, 分配方案详见上述分析

$$n = \begin{cases} k & 1 \leq k \leq 200 \\ 200 + 2^a & a \in \mathbb{N} \end{cases}$$

Problem 4 利用循环不变量, 证明插入排序算法的正确性

```
// Algorithm: InsertionSort
for i = 2 to n
    key = A[i];
    j = i - 1;
    while (j > 0 && A[j] > key)
        A[j+1] = A[j];
        j--;
    end while
    A[j+1] = key;
end for
```

- for循环开始时, 中间值 $A[1..i-1]$ 包含了初值 $A[1..i-1]$ 的所有元素且已排好序, 为循环不变量
 1. $i = 2$, 此时 $A[1..i-1]$ 只有 $A[1]$, 显然有序, 成立
 2. $2 < i \leq n$ 设开始时中间值 $A[1..i-1]$ 有序, 则while循环中比当前值 $A[i]$ 大的元素均右移1位, 之后将 $A[i]$ 插入空位, 保证了中间值 $A[1..i]$ 来自初值 $A[1..i]$ 的元素且保持有序性
 3. $i = n + 1$ 循环结束时 $i - 1 = n$, 则结果 $A[1..n]$ 为初值 $A[1..n]$ 的所有元素, 且已排序

1.2 Week 2-20250227

Problem 1 证明: $\log n = o(n^k)$, n 为常数

- $\log n = c \ln n$, c 为常数
- $\therefore \lim_{n \rightarrow \infty} \frac{\log n}{n^k} = \lim_{n \rightarrow \infty} \frac{c \ln n}{n^k} = \lim_{n \rightarrow \infty} \frac{\frac{c}{n}}{k \cdot n^{k-1}} = \lim_{n \rightarrow \infty} \frac{c}{k \cdot n^k} = 0$
- $\therefore \log n = o(n^k)$

Problem 2 举例单调递增函数 $f(n), g(n)$, 使得 $f(n) \neq O(g(n)), g(n) \neq O(f(n))$

Solution 1

$$f(n) = \begin{cases} n! & n = 2k + 1, k \in \mathbb{N} \\ (n+1)! & n = 2k, k \in \mathbb{N} \end{cases}, g(n) = \begin{cases} (n+1)! & n = 2k + 1, k \in \mathbb{N} \\ n! & n = 2k, k \in \mathbb{N} \end{cases}$$

Solution 2

$$f(n) = n^{n+\sin n}, g(n) = n^{n+\cos n}$$

Problem 3 证明 $\sum_{i=1}^n \frac{1}{i} = \Theta(\log n)$

- $\sum_{i=1}^n \frac{1}{i} \leq 1 + \int_1^n \frac{1}{x} dx = 1 + \log n$
- $\sum_{i=1}^n \frac{1}{i} \geq \int_1^n \frac{1}{x} dx = \log n$
- $\therefore \sum_{i=1}^n \frac{1}{i} = \Theta(\log n)$

Problem 4 将以下函数按增长速度排序

$$f_1(n) = n^{\frac{1}{3}}, f_2(n) = \log n, f_3(n) = 2^{\sqrt{n}}$$

- 取对数

$$- \log f_1(n) = \frac{1}{3} \log n = \Theta(\log n)$$

$$- \log f_2(n) = \log \log n = \Theta(\log \log n)$$

$$- \log f_3(n) = \sqrt{n} \log 2 = \Theta(\sqrt{n})$$

- $\therefore$ 增长速度 $f_2 \prec f_1 \prec f_3$

2 Homework 2

2.1 Week 3–20250306

Problem 1 $T(n) = n + \sum_{i=1}^{n-1} T(i)$, $T(1) = 1$, 求 $T(n)$ 的非递推解析式

$$\begin{aligned} T(n+1) &= n+1 + \sum_{i=1}^n T(i) \\ &= T(n) + 1 + n + \sum_{i=1}^{n-1} T(i) \\ &= 2T(n) + 1 \end{aligned}$$

- $\therefore T(n+1) + 1 = 2(T(n) + 1)$, $\{T(n) + 1\}$ 是首项为 $T(1) + 1 = 2$, 公比为 2 的等比数列
- $\therefore T(n) + 1 = 2^n$, $T(n) = 2^n - 1$

Problem 2

$$T(n) = \begin{cases} T\left(\frac{n}{2}\right) + 1 & n = 2k, k \in \mathbb{N}_+ \\ 2T\left(\frac{n-1}{2}\right) & n = 2k+1, k \in \mathbb{N}_+ \end{cases}$$

其中 $T(1) = 1$

1. 证明 $n = 2^k$ 时, $T(n) = O(\log n)$
2. 证明存在无穷集合 X , 满足 $n \in X$ 时 $T(n) = \Omega(n)$
3. 由1.和2.能得出什么结论?

1. $T(2^k) = T(2^{k-1}) + 1 \therefore T(2^k) = k + 1$
 $\therefore T(n) = k + 1 = \log_2 n + 1 = O(\log n)$
2. 令 $X = \{n | n = 2^k - 1, k \in \mathbb{N}_+\}$
 $T(2^k - 1) = 2T\left(\frac{2^k - 1 - 1}{2}\right) = 2T(2^{k-1} - 1)$
 $\therefore T(2^k - 1) = 2^{k-1}, T(n) = 2^{\log_2(n+1)-1} = \frac{n+1}{2} = \Omega(n)$
3. 为算法分析方便，我们经常假设 $n = 2^k$ ，并简单地加以推广，但这并不总是正确

Problem 3 设计算法求给定序列的最大连续子列积

在已知已扫描过的部分的最大连续子列积、最大后缀子列积的基础上，还要保存并更新最小连续子列积，保证后续扫描到负数时的正确性

Problem 4 给定 $nums[1..n]$ 表示股票在第 $1 \sim n$ 天的价格，若买入、卖出各1次，设计算法求最大收益

Solution 1 申请的数组 $diff[1..n-1]$, $diff[i]$ 保存第 $i+1$ 天与第 i 天的价格差，在 $diff[n-1]$ 的基础上求最大连续子列和，时间复杂度 $O(n)$

Solution 2 申请2个数组 $A[n]$, $B[n]$

- $A[i]$ 保存前 i 天的最低价格，可扫描 $nums[1..n]$ 初始化 $A[]$
- $B[i]$ 保存后 $n-i$ 天的最低价格，可扫描 $nums[n..1]$ 初始化 $B[]$

然后对 $i = 1 \text{ to } n$ 计算 $B[i] - A[i]$ 的最大值，时间复杂度 $O(n)$

Problem 5 讨论汉诺塔问题

1. 给出汉诺塔问题的移动算法
2. 已知各个圆盘状态（在给出的各个圆盘在柱子的分布中，小盘一定在大盘上方），求出这是1.中算法的步数（若该状态在1.中不会出现，则返回-1）

```
1. //Algorithm: Hanoi
   //Input: n, src, mid, dst
   //Output: Each step
   Hanoi(n, src, mid, dst)
   {
       if (n==1) step = src + "->" + dst;
```

```

    else
    {
        Hanoi(n-1, src , dst , mid);
        step = src + "->" + dst;
        Hanoi(n-1, mid, src , dst);
    }
}

```

2. 从最大盘开始考虑

- 如果最大盘在 **src**, 则问题简化为: 将 $1..n-1$ 从 **src** 移动到 **mid** 的过程, 即求当前状态在 $\text{Hanoi}(n-1, \text{src}, \text{dst}, \text{mid})$; 中的步数
- 如果最大盘在 **dst**, 则问题简化为: 已经将 $1..n-1$ 从 **src** 移动到 **mid**, 共花费了 2^{n-1} 步, 只要求当前状态在 $\text{Hanoi}(n-1, \text{mid}, \text{src}, \text{dst})$; 中的步数
- 如果最大盘在 **mid**, 则不可能出现该情况, 返回 -1

//Algorithm: # of steps in Hanoi

//Input: Condition

//Output: # of steps

Step(n, m, src , mid, dst)

```

{
    if (m == 0) return 0;
    if (n-m+1 at mid) return -1;
    if (n-m+1 at src) return Step(n, m-1, src , dst , mid);
    if (n-m+1 at dst)
    {
        tmp = Step(n, m-1, mid, src , dst);
        if (tmp == -1) return -1;
        else return  $2^{(m-1)} + \text{tmp}$ ;
    }
}

```

Problem 6 $A[1..n]$ 为整数数组, 构造 $B[1..n]$, 其中 $B[i] = \max_{i \leq j \leq n} A[i]A[j]$, 求构造 $B[1..n]$ 的算法及其时间复杂度

扫描 $A[n..1]$ ，维护最大正数和最小负数，遇到 $A[i]$ 时根据 $A[i]$ 的正负情况，与前述两数之一相乘以构造 $B[i]$ ，时间复杂度 $O(n)$

2.2 Week 4–20250313

Problem 1 给定单调递增数列，设计高效算法求出绝对值最小的元素

- 若第1个数为正，则返回第1个数
- 若第 n 个数为负，则返回第 n 个数
- 否则二分查找，并附加中间值为0直接返回的机制
- 时间复杂度 $O(\log n)$

Problem 2 给定数组 $A[1..n]$ ，相邻元素最多相差 1， $A[1] = x$ ， $A[n] = y$ ， $x \leq z \leq y$ ，求 z 在 $A[]$ 中出现的任意1个位置，设计算法并给出时间复杂度

可类比数学中连续函数的性质和零点存在定理，二分查找保证找到1个解，时间复杂度 $O(\log n)$

Problem 3 我们知道，归并排序时间复杂度 $O(n \log n)$ ，插入排序时间复杂度 $O(n^2)$ ；但在数据规模足够小时，由于常数因素，插入排序比归并排序更快

考虑在数组被分割到足够小时，用插入代替归并以优化算法，对长度为 k 的 $\frac{n}{k}$ 个子序列使用归并合并

1. 证明：对长度为 k 的 $\frac{n}{k}$ 个子序列使用插入排序的时间复杂度为 $O(nk)$
2. 设计算法将子序列归并，要求时间复杂度 $O(n \log \frac{n}{k})$
3. 由1.和2.可知总复杂度 $O(nk + n \log \frac{n}{k})$ ，令 $O(nk + n \log \frac{n}{k}) = O(n \log n)$ 的最大 k 是多少？用 $O(f(n))$ 的形式表示
4. 实际编程实现算法时，应如何选择 k ？

1.
 - 单个序列 $O(k^2)$
 - 总复杂度 $O\left(\frac{n}{k}\right) O(k^2) = O(nk)$
2.
 - 解法1：成对地将子序列合并，每一级均为 $O(n)$ ，共 $\left\lceil \log \frac{n}{k} \right\rceil$ 级，总复杂度 $O\left(n \log \frac{n}{k}\right)$

- 解法2: 堆排序: 维护最大二叉堆, 堆中元素记录待排序的数据和数据所属的队列, 每次出堆时, 从出堆元素所属子列中选择下个元素入堆, 堆操作 $\left\lceil \log \frac{n}{k} \right\rceil$, 共 n 个元素, 总复杂度 $O\left(n \log \frac{n}{k}\right)$
3. $k = O(\log n)$
- 显然 $k = O(1)$ 时 $n \log \frac{n}{k} = O(n \log n)$
 - 若 $\log n = o(k)$, 则 $n \log n = o(nk)$, 不成立
 - 故 k 的最高阶数为 $O(\log n)$
4. k 应为插入排序比归并排序更快的最大数组规模

Problem 4 有 n 个不同大小的螺母和 n 个不同大小的螺栓, 且能恰好完全配对。螺母和螺母不能比较大小, 螺栓和螺栓不能比较大小。螺栓与螺母尝试匹配时, 若松动, 说明螺栓较小; 若无法插入, 说明螺母较小; 否则说明螺母与螺栓恰好匹配。设计匹配全部螺母和螺栓的方法, 并给出比较次数的期望阶数

根据快速排序思想, 用1个螺母与全部螺栓尝试匹配, 可将螺母分为较小、较大、恰好匹配3组 (其中恰好匹配只有1个); 再用恰好匹配的螺母与所有螺栓尝试匹配, 可将螺栓分为较小、较大、恰好匹配3组 (其中恰好匹配只有1个); 之后较小的螺母与较小的螺栓按同样方法匹配, 较大的螺母与较大的螺栓匹配 (递归)

比较次数 $T(n)$ 满足 $T(n) = \frac{\sum_{s=0}^{n-1} (T(s) + T(n-1-s))}{n} + 2n$, 与快速排序类似, 所以 $T(n) = O(n \log n)$

Problem 5 设 $A[1..n]$ 是整数 $1..n$ 的任意排列, 若 $i < j$ 时 $A[i] > A[j]$, 称 $\langle i, j \rangle$ 为 A 中的1个逆序对

1. 列出 $A = \{2, 3, 5, 4, 1\}$ 中的逆序对
2. $A[1..n]$ 最多包含多少个逆序对? 此时 $A[1..n]$ 的数据分布情况如何?
3. 设计算法求 $A[1..n]$ 中的逆序对数目, 要求时间复杂度 $O(n \log n)$

1. $\langle 1, 5 \rangle, \langle 2, 5 \rangle, \langle 3, 5 \rangle, \langle 4, 5 \rangle, \langle 3, 4 \rangle$

注意: 逆序对用其下标表示

2. $\frac{n(n-1)}{2}$ 个; $A[1..n] = [n..1]$

3. 根据归并排序思想, 将数组元素从大到小排序

- size == 1 直接返回
- size == 2
 - 若 $A[i-1] < A[i]$ 则交换两元素

– 否则不交换元素，但逆序对数 +1

- **size** >= 3 归并过程：前半数组中的元素出队时，逆序对数 + 后半数组中剩余元素的数量

时间复杂度：给归并排序每个 $O(1)$ 步骤附加了一个 $O(1)$ 步骤，最终复杂度 $O((1+O(1))n \log n) = O(n \log n)$

Problem 6 证明如下排序算法的时间复杂性，并求其时间复杂度

```
//Algorithm: 3-sort
//Input: A[1..n]
//Output: A[1..n] in order
three_sort(Array[], left, right)
if (left + 1 == right)
    if (Array[left] > Array[right])
        swap(Array[left], Array[right])
if (left + 1 <= right)
    k = (right-left+1)/3
    three_sort(Array[], left, right-k)
    three_sort(Array[], left+k, right)
    three_sort(Array[], left, right-k)
```

- 数组只有 1 个元素，则什么也不做，确保数组有序
- 数组只有 2 个元素，则条件交换，确保数组有序
- 数组包含 ≥ 3 个元素：
 - $\because 1 \leq k \leq \frac{right-left+1}{3}$
 $\therefore left < right-k, left+k < right$, 确保参数合法
 - 考虑最大的 $\left\lfloor \frac{n}{3} \right\rfloor$ 个元素
 - * 若其初始位置位于最右侧的 $\left\lfloor \frac{n}{3} \right\rfloor$ ，则第 1 轮 3-sort 不影响其位置
 - * 否则，第 1 轮 3-sort 后一定会出现在中间 $\left\lfloor \frac{n}{3} \right\rfloor$
 - * 无论如何，第 2 轮 3-sort 后一定会被放在最右侧的 $\left\lfloor \frac{n}{3} \right\rfloor$ ，确保数组有序
 - 第 3 轮‘3-sort’一定能将其余 $\left\lfloor \frac{2n}{3} \right\rfloor$ 排序
- 求时间复杂度：

$$T(n) = 3T\left(\frac{2n}{3}\right) + O(1) \Rightarrow T(n) = O(n^{\log_{\frac{3}{2}} 3})$$

- 优于 $O(n^2)$ 的排序算法如插入排序、冒泡排序，但劣于 $O(n \log n)$ 的排序算法如归并排序、快速排序

3 Homework 3

3.1 Week 5–20250320

Problem 1 设 $A[1..s]$ 和 $B[1..t]$ ($s < t$) 均有序，设计算法求第 k 小的数，并给出时间复杂度

- $k \leq s$ ，直接求 $A[1..k]$ 和 $B[1..k]$ 的中位数即可
- $k > t$
 - $A[k-t] \geq B[t]$ ，返回 $A[k-t]$
 - $B[k-s] \geq A[s]$ ，返回 $B[k-s]$
 - 否则，求 $A[k-t+1..s]$ 和 $B[k-s+1..t]$ 的中位数
 - * $A[k-t] < B[t]$ ，则比 $A[k-t]$ 小的数最多只可能有 $(k-t-1)+(t-1) = k-2$ 个，所以 $A[1..k-t]$ 一定不是解
 - * $B[k-s] < A[s]$ ，则比 $B[k-s]$ 小的数最多只可能有 $(k-s-1)+(s-1) = k-2$ 个，所以 $B[1..k-s]$ 一定不是解
 - * 可排除 $(k-t)+(k-s)$ 个数，因此只要求剩余数中第 $k-(k-s)-(k-t) = s+t$ 小的数，即上述2个等长数组的中位数
- $s < k \leq t$
 - $B[k-s] \geq A[s]$ ，返回 $B[k-s]$
 - 否则，求 $A[1..s]$ 和 $B[k-s+1..k]$ 的中位数
 - $B[k-s] < A[s]$ ，则比 $B[k-s]$ 小的数最多只可能有 $(s-1)+(k-s-1) = k-2$ 个，所以 $B[1..k-s]$ 一定不是解
 - 可排除 $k-s$ 个数，因此只要求剩余数中第 $k-s$ 小的数，即上述2个等长数组的中位数
- 时间复杂度： $O(1)$ 预处理， $O(\log s)$ 求2个等长数组（规模不超过 s ）的中位数，总复杂度 $O(\log s)$

Problem 2

1. 求2个字符串的最长公共子序列（不要求子序列在原序列中连续，仅要求子序列放回原序列的过程中顺序保持不变）的长度
2. 利用最长公共子序列，求2个字符串的最短公共超序列的长度

1. 动态规划

- 初值: $dp[i, 0] = dp[0, j] = 0$
- 迭代:

$$dp[i, j] = \begin{cases} 1 + dp[i-1, j-1] & \text{if}(str1[i] = str2[j]) \\ \max\{dp[i-1, j], dp[i, j-1]\} & \text{if}(str1[i] \neq str2[j]) \end{cases}$$

2. 最短公共超序列长度 = $\text{strlen}(\text{str1}) + \text{strlen}(\text{str2}) + \text{LCS}$, 其中 LCS 为最长公共子序列长度

Problem 3 2^k 位选手要参加单循环比赛 (任意2位选手之间都要进行1场比赛)

1. 设计算法以安排赛程
2. 若任意2位选手的比赛结果均可查, 设计算法求序列 $P[1..n]$, 使 $\forall i \in [1, n-1] \cap \mathbb{Z}, P[i] \text{ beats } P[i+1]$

1. 数学归纳法

- $n = 2$

1	2
2	1

- $n = 4$

1	2	3	4
2	1	4	3
3	4	1	2
4	3	2	1

- $n = 8$

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

- 第 $2 \sim 2^k$ 行分别为 $2^k - 1$ 轮比赛内第 1 行选手的对手

- k 每次增加时, 按如下方式处理:

$$M_{k+1} = \begin{bmatrix} M_k & M'_k \\ M'_k & M_k \end{bmatrix}$$

其中 M'_k 为 M_k 中每个元素 $+2^{k-1}$ 后的矩阵

2. 直接进行归并排序即可

- 设归并前2个子数组均满足 $\forall i \in [1, k-1] \cap \mathbb{Z}, P[i] \text{ beats } P[i+1]$, 则归并时每次取队首2位选手的胜者出队即可
- 由于任意1位选手出队后, 与该选手同队列的后1名选手被出队选手击败, 另一队列的队首选手也被出队选手击败, 保证了无论下次哪个队列的队首选手出队, 都会被其在合并队列的前1位选手击败
- 实际上任意基于比较的排序算法都可实现

Problem 4 $m \times n$ 的矩阵 A , 满足 $A[i, j] \leq A[i+1, j] \wedge A[i, j] \leq A[i, j+1]$, 且允许某些位置为 ∞ , 例如

2	3	9	12
4	8	14	∞
5	∞	∞	∞
16	∞	∞	∞

设计满足时间复杂性要求的算法:

1. $O(m+n)$ 删除最小元素
2. $O(m+n)$ 插入元素
3. $O(n^3)$ 排序 n^2 个元素
4. $O(m+n)$ 元素查找

1. 删除左上角元素, 对于空位:

- 若其右邻元素和下邻元素均为 ∞ , 则补充 ∞ 并退出
- 否则, 选右邻元素和下邻元素中较小的元素与空位交换

2. 将元素置于左上角

- 若新元素小于右邻元素和下邻元素则退出
- 否则选择其右邻元素或下邻元素中较小的元素与其交换, 并将原右邻元素或下邻元素视为新元素

3. 维护 $n \times n$ 的杨氏矩阵, 依次插入所有元素, 再依次删除最小元素

4. 从右上角元素开始

- 若待查元素大于矩阵当前元素, 则当前元素向下移动

- 若待查元素小于矩阵当前元素，则当前元素向左移动
- 若待查元素等于矩阵当前元素，则返回查找结果
- 若检查过程超出了左边界或下边界，则查找失败

Problem 5 邮局选址问题可建模为：给定 n 个点 $P[1..n]$ 及其权值 $w[1..n]$ ，选择点 Q

使 $\sum_{i=1}^n w_i |Q, P[i]|$ 最小

1. 证明加权中位数即为所求解
2. 设二维 *Manhattan* 距离 $|P, Q| = |P_x, Q_x| + |P_y, Q_y|$ ，如何选择邮局地址？

1. 设 Q 就在加权中位数位置

- 满足 $\sum w_{left} \leq \frac{1}{2}$, $\sum w_{right} \leq \frac{1}{2}$ ，其中 $left, right$ 分别是 Q 左侧的所有点和右侧的所有点
 - 此时若 Q 向左移动 d ，则加权距离和的增加量 $d \sum w_{right}$ ，减少量 $d \sum w_{left}$ ，增加量一定不小于减少量；且随着 Q 向左移动， $\sum w_{right}$ 逐渐增大， $\sum w_{left}$ 逐渐减小，不可能增加量大于减少量
 - Q 向右移动同理
- 因此加权中位数就是邮局选址最优解

2. 分别求出 x, y 方向的加权中位数即为邮局选址

Problem 6 对文本进行 *Huffman* 编码，字符集的字符种类数为 256，最高频字符的频数不超过最低频字符频数的 2 倍

证明：*Huffman* 编码不会比定长 8 位编码更高效

设二叉树的根节点为第 0 层，则定长编码可看作 8 层二叉树的叶子结点

由二叉树的性质，至少减少 2 个 8 层结点才能将 1 个 7 层结点变为叶子结点（即产生 1 个字符的 *Huffman* 编码）

且由于定长编码的二叉树的第 8 层是满的，减少的 2 个 8 层结点至少要放在第 9 层才能保证有 256 个叶子结点

设 f 为频数，则变化过程的编码长度变化量为 $f_1 + f_2 - f_0$ ，而 $f_0 \leq 2f_1$, $f_0 \leq 2f_2$ ，有 $f_0 \leq f_1 + f_2$ ，因此编码总长度不会减少

3.2 Week 6–20250327

Problem 1 $G = (V, E)$ 为无向图，每个顶点的度数均为偶数

设计线性时间算法，为每条边赋予方向，使 G 变为有向图，且每个顶点的入度等于出度

易知 G 是欧拉图，每条边沿任意欧拉回路的方向即可

Problem 2 连连看游戏中规定：连线拐弯次数小于等于 2 则可消去

1. 设计算法判断给定 2 个图形能否消去
2. 不考虑消去规则，设计算法求 2 个图形连线的最小拐弯次数

1. 设给定的 2 个图形分别为 A, B

- 分别过 A, B 作尽可能长的水平线段，检查水平线段之间是否有竖直连线
- 分别过 A, B 作尽可能长的竖直线段，检查竖直线段之间是否有水平连线

2. 变种DFS:

- 以‘ A ’点作尽可能长的水平和竖直线段，并标记 0
- 以标记为 0 的点作尽可能长的、与其和‘ A ’的连线垂直的线段，并标记 1
- 不断翻转方向，并递增标记值，直到遇到‘ B ’即可

Problem 3 证明：无向图中必存在某点，删除该点不影响图的连通性

DFS的叶结点

Problem 4 $G = (V, E)$ 是有向图，边的权重满足 $\forall e \in E, w(e) \in \{1, 2, 3\}$

设计算法求解单源最短路径问题，要求时间复杂度 $O(|V| + |E|)$

根据边的权值 i ，将边展开为 i 条边，并向其中添加 $i - 1$ 个结点，然后求无权图的单源最短路径即可（BFS）

Problem 5 G 是无回路的有向图，其路径最多含有 k 条边

设计线性时间算法，将 $V(G)$ 划分为至多 $k + 1$ 个组，使得每个组内任意 2 个结点间均不存在路径

拓扑排序扩展：维护 2 个队列，每次移除其中一个队列内的所有结点并归为一组，移除过程中发现的度数为 0 的结点加入到另一队列

4 Homework 4

4.1 Week 7–20250403

Problem 1 给定无向图 $G = (V, E)$ 及 $a, b, c \in E$, 设计线性时间算法, 判断 G 中是否存在包含 a, b 但不包含 c 的回路

将 c 去掉, 之后求 $G - c$ 的双连通分支划分, 判断 a, b 是否属于同一个双连通分支

Problem 2 设计线性时间算法求树的最大匹配

每次找到叶子结点时, 将它与它的父结点匹配, 并从树中删除这 2 个结点, 得到一个或多个规模较小的树递归求解

Problem 3 设计线性时间算法求树的最小顶点覆盖

每次找到叶子结点时, 将其父结点加入顶点覆盖中, 并删除加入顶点覆盖中结点关联的所有边, 得到一个或多个规模较小的树递归求解

4.2 Week 8–20250410

Problem 1 P 是包含在矩形 R 中的一个简单多边形, q 为 R 中任意一点
设计算法, 求以 q 为顶点的一条射线, 使其与 P 交点最少, 并给出时间复杂度

- 将 q 与多边形 P 各顶点连线, 按旋转角排序多边形 P 的各顶点, 初始化射线旋转角为 0 时的交点个数, 按上述排序生成事件序列并依次处理, 每次遇到顶点时
 - 若 2 条相关边都在扫描线顺时针一侧, 交点数 -2
 - 若 2 条相关边都在扫描线逆时针一侧, 交点数 $+2$
 - 若 2 条相关边在扫描线异侧, 交点数不变扫描过程中维护交点个数最小值, 扫描一圈后返回
- 时间复杂度:
 - 排序: $O(n \log n)$
 - 初始化: $O(n)$
 - 事件处理: 单个事件 $O(1)$, 全部事件 $O(n)$总时间复杂度 $O(n \log n)$

Problem 2 求平面上一组点中任意 2 个顶点之间距离的最大值，设计算法并给出时间复杂度

旋转卡壳法：首先计算凸包，找到凸包上 x 坐标最小的点 p_0 ，以及凸包上距离 p_0 最远的点 q_0 ，比较 $\langle \overrightarrow{q_i p_i}, \overrightarrow{q_i p_{i+1}} \rangle$ 与 $\langle \overrightarrow{p_i q_i}, \overrightarrow{p_i q_{i+1}} \rangle$ 的大小关系，选择较大者，更新 p_i 或 q_i ，旋转过程中维护距离最大值，旋转一圈后返回

时间复杂度：

- 计算凸包： $O(n \log n)$
- 初始化： $O(n)$
- 旋转卡壳： $O(n)$
- 总时间复杂度 $O(n \log n)$

Problem 3 为Problem 2设计近似算法如下：只计算某个顶点到其他顶点距离的最大值，并以此作为问题的近似解

证明：该算法得到的结果不小于最优值的一半

设最大距离准确解为 d_{ab} ，近似算法中点 p_x 与其他顶点距离的最大值为 d_{xy}

根据近似算法原理有： $d_{ax} \leq d_{xy}$ ， $d_{bx} \leq d_{xy}$

根据三角形两边之和大于第三边有： $d_{ax} + d_{bx} \geq d_{ab}$

$$\therefore 2d_{xy} \geq d_{ax} + d_{bx} \geq d_{ab}, \quad d_{xy} \geq \frac{1}{2}d_{ab}$$

$\therefore$ 该算法得到的结果不会小于最优值的一半

Problem 4 给定 n 种液体都只含有 A, B 2 种成分，含量分别为 $(a_i, b_i), i = 1, 2, \dots, n$ ，目标液体 T 中 A, B 的含量分别为 a, b

设计算法判断 T 能否由给定 n 种液体配制

判断点 (a, b) 是否在 (a_i, b_i) 的凸包中即可，时间复杂度 $O(n \log n)$

拓展：给出一种 $O(n)$ 算法如下

以 (a, b) 为原点， (a_1, b_1) 为 x 轴建立二维坐标系，找出旋转角与 180° 相邻的 2 个点 P, Q ，判断直线 PQ 的截距是否小于 0 即可

该算法理论上计算复杂性较低，但实现过程中计算步骤较复杂

5 Homework 5

5.1 Week 9–20250417

Problem 1 给定平面上 n 个点 (x_i, y_i) , $i = 1, 2, 3, \dots, n$, 极大点定义为: 若 (x_i, y_i) 满足 $\forall j \neq i, (x_j < x_i) \vee (y_j < y_i)$, 则 (x_i, y_i) 为极大点

设计算法求给定点集的极大点, 并给出时间复杂度

Solution 1

按照 x 坐标从大到小排序 (x 坐标相同的点, 按照 y 坐标从大到小排序) 之后扫描线从右向左扫描, 过程中维护 y 坐标的最大值 (初值尽可能小), 若 y 坐标最大值有更新, 则将该点加入解集中

时间复杂度取决于排序算法, 最快为 $O(n \log n)$

Solution 2

求出凸包, 之后找到凸包上 x 坐标最大的点和 y 坐标最大的点 (如有多个, 则取另一个坐标尽可能大者), 从 x 坐标最大的点出发, 逆时针前进到 y 坐标最大的点, 路径上的所有点即为极大点

时间复杂度取决于求凸包算法, 最快为 $O(n \log n)$

Problem 2

1. 求凸 $n + 2$ 多边形不同三角划分种类数 (写出递推式即可)
2. 设计算法, 求三角划分对角线长度和的最小值

1. 选择 1 条边, 以及不与该边相关联的任意 1 个顶点, 可将多边形划分为 3 部分, 这 3 部分分别**拥有原多边形边**的数量为 $i, n - 1 - i, 1$

设 C_n 表示将凸 $n + 2$ 多边形划分为 n 个三角形的种类数, 则有递推式

$$C_0 = 1 \quad C_n = \sum_{i=1}^n C_{i-1} C_{n-i}$$

C_n 为卡特兰数, 推导过程详见 CS2304 计算机科学中的数学基础第2章组合数学第3节生成函数, 解析解为

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

2. 动态规划

- $dp[i, j]$ 为 v_i 到 v_j 的所有边加上边 (v_j, v_i) 组成的多边形的三角划分中最小对角线长度和 (i 到 j 的部分按照 $1, 2, \dots, n$ 的循环队列增序依次排列)

- 初值:

$$dp[i, j] = \begin{cases} 0 & i = j \vee (i + 1) \% n = j \\ |v_i, v_j| & (i + 2) \% n = j \end{cases}$$

- 迭代:

$$dp[i, j] = \min_k \{dp[i, k] + dp[k, j] + |v_i, v_k| + |v_k, v_j|\}$$

$$k \in \begin{cases} (i, j) & i < j \\ (i, n] \cup [1, j) & i > j \end{cases} \wedge k \in \mathbb{N}_+$$

- 问题解: 满足 $(j + 1) \% n = i$ 的 $dp[i, j]$ 均为问题解

Problem 3 给定平面内 n 条线段, 设计算法判断其中有无线段间交点, 要求时间复杂度 $O(n \log n)$

- 算法与求 n 条线段所有交点的算法相同, 只是遇到交点就结束
- 由于遇到交点就结束, 所以时间复杂度有所降低:
 - 事件点数 $\leq 2n = O(n)$
 - 单个事件状态数 $\leq \log 2n = O(\log n)$
 - 总时间复杂度 $O(n \log n)$

Problem 4 利用扫描线算法求解最近点对问题

先将各点按 x 坐标从小到大 (x 坐标相同的点按 y 坐标从小到大排序)

维护扫描过程中的最近距离, 初始化为 $|v_1, v_2|$

扫描每个点时的事件处理也为扫描线算法:

- 从左向右扫描各个点, 并维护最小距离
- 若扫描过程中扫描线与始点的 x 坐标差已大于当前最小距离, 则提前结束事件点

Problem 5 给定 $A[1..n]$, $A[i]$ 表示 $x = i$ 处木板高度, 任意 2 块木板与 x 轴可构成容器, 设计算法求单个容器的最大容积

初始化 $left = 1, right = n, res = \min\{A[1], A[n]\} \times (n - 1)$

之后对 $A[left]$ 或 $A[right]$ 作出处理, 处理步骤为:

- 选择 $A[left]$ 或 $A[right]$ 中较小的一个 $A[move]$

- 暂存该侧木板高度的当前值 tmp ，每次向中间 ($left++$ 或 $right--$) 移动 1，并比较移动后的 $A[move]$ 与 tmp
 - 若 $A[move] \leq tmp$ ，则继续移动
 - 否则重新计算 $\min\{A[left], A[right]\} \times (n-1)$ 并更新 res
- 直到 $left = right$
- 时间复杂度：
 - 移动次数： $n = O(n)$
 - 事件点处理： $O(1)$
 - 总时间复杂度 $O(n)$

5.2 Week 10–20250424

Problem 1 海报墙由高为 $h[1..n]$ 的 n 块木板组成，木板的宽度均为 1

求海报墙上能粘贴矩形海报的最大面积，要求海报的边必须与木板侧边平行或垂直，给出算法及其时间复杂度

依次求 $left[1..n]$, $right[1..n]$ ，其含义为：从木板 i 开始，若左侧或右侧木板不低于木板 i ，则向左或向右移动，直到遇到低于木板 i 的木板，将其下标对应保存到 $left[i]$, $right[i]$

维护方式：设立单调递增栈 s ，其元素存放木板下标

- 从左向右扫描木板
 - 若 $h[i] > h[s.top]$ ，则 $left[i]$ 为 $s.top + 1$ ，并将 i 入栈
 - 若 $h[i] \leq h[s.top]$ ，则 $s.top$ 出栈，并再次比较 $h[i]$ 与 $h[s.top]$
- 清空上述过程的栈
- 从右向左扫描木板
 - 若 $h[i] > h[s.top]$ ，则 $right[i]$ 为 $s.top - 1$ ，并将 i 入栈
 - 若 $h[i] \leq h[s.top]$ ，则 $s.top$ 出栈，并再次比较 $h[i]$ 与 $h[s.top]$
- 为便于实现上述维护，可设 $h[0] = h[n+1] = 0$

最后，遍历数组并求出 $\max_{1 \leq i \leq n} (right[i] - left[i] + 1) \times h[i]$ 即为问题的解

由于每个元素入栈、出栈最多 1 次，所以维护 $left[1..n]$, $right[1..n]$ 的时间复杂度均为 $O(n)$ ，总时间复杂度 $O(n)$

Problem 2 给定数组 $Enve[1..n]$ ， $Enve[i] = (w_i, h_i)$ 表示第 i 个信封的宽度和高度；当信封 a 的宽度和高度都比信封 b 大时，可将信封 b 装入信封 a ，此时还允许信封 a 内含其他信封、信封 b 装入其他信封（即“套娃”）

设计算法求最多能有多少个信封嵌套在一起

将‘Enve[1..n]’按照 w_i 升序 (w_i 相同的按 h_i 降序) 排序, 之后求 $h[1..n]$ 序列的不必连续的最长递增子序列即可

Problem 3 求 $n!$ 包含质因子 p 的次数, 设计算法并给出时间复杂度

初始化 $res = 0, k = 0$, 运行如下步骤

$$while (n \geq p^k) \quad res = res + \left\lfloor \frac{n}{p^k} \right\rfloor, \quad k = k + 1$$

最后得到 res 即为问题解

时间复杂度 $O(\log_p n) = O(\log n)$, 由于输入规模 $O(\log n)$, 所以该算法为线性时间

Problem 4 证明: $\forall n \in \mathbb{N} \wedge n \geq 2$, n 可表示为至多 $\log_2 n$ 个 *Fibonacci* 数之和, 并设计算法求上述表示方式

易知 $Fib(n) > Fib(n-1) > Fib(n-2)$

$\therefore Fib(n) = Fib(n-1) + Fib(n-2)$

$\therefore Fib(n) < Fib(n-1)$

$\therefore$ 小于 n 的最大 *Fibonacci* 数 $Fib(k) > \frac{n}{2}$

设 $< \frac{n}{2}$ 的整数可用 $< \log_2 \frac{n}{2} = \log_2 n - 1$ 个 *Fibonacci* 数之和表示, 则 $n - Fib(k) < \frac{n}{2}$ 个 *Fibonacci* 数之和表示

$\therefore n$ 可用上述 $\log_2 n - 1$ 个数加上 $Fib(k)$ 表示

由数学归纳法可知成立, 算法设计与证明过程类似

Problem 5 用线性时间算法求 *Fibonacci* 数列的第 n 项即 $Fib(n)$, 假设不支持浮点数操作

$$\begin{aligned} \begin{pmatrix} Fib(n) \\ Fib(n-1) \end{pmatrix} &= \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} Fib(n-1) \\ Fib(n-2) \end{pmatrix} \\ &= \dots \\ &= \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-2} \begin{pmatrix} Fib(2) \\ Fib(1) \end{pmatrix} \end{aligned}$$

关键在于求 $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-2}$ ，可用分治法计算
 时间复杂度 $O(\log n)$ ，与输入规模同阶，为线性时间

6 Homework 6

6.1 Week 12-20250508

Problem 1 使用不包含溢出修复功能的程序语言，如何在整型数范围内计算 $\binom{n}{k}$ ，要求计算范围尽可能大

使用递推关系 $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ 进行计算

Problem 2 P 是一个十进制整型数，将 P 划分为 k 段并求各段的乘积，称为 P 的一个 k -乘积

1. 求 1234 的所有 2-乘积
2. 求任意整数 P 的最大 k -乘积 ($P > 10^{k-1}$)

1. $1 \times 234 = 234$, $12 \times 34 = 408$, $123 \times 4 = 492$
2. 动态规划: $dp[i, j]$ 表示 P 前 i 位的最大 j -乘积

- 初值:
 - $dp[i, 1]$ 为 P 的前 i 位组成的整数
 - $dp[i, j] = 0, i < j$

- 迭代:

$$dp[i, j] = \max_{1 \leq k \leq i-1} \{dp[k, j-1] \times \overline{k+1..i}\}$$

其中 $\overline{k+1..i}$ 表示 P 的第 $k+1$ 高位到第 i 高位组成的整数

- 问题解: $dp[n, k]$

Problem 3 求 $1 \oplus 2 \oplus 3 \oplus \dots \oplus N$ 的值，其中 $\oplus$ 表示异或，所有数字实际均用二进制码表示， N 为输入，要求计算量尽可能小

- 记 $f(N) = 1 \oplus 2 \oplus 3 \oplus \dots \oplus N$, 则有:
 - 若 $N \% 4 == 0$, 则 $f(N) = N$
 - 若 $N \% 4 == 1$, 则 $f(N) = 1$
 - 若 $N \% 4 == 2$, 则 $f(N) = N + 1$
 - 若 $N \% 4 == 3$, 则 $f(N) = 0$
- 时空复杂度均为 $O(1)$

Problem 4 将正整数 n 表示为 $n = n_1 + n_2 + \dots + n_k (n_1 \geq n_2 \geq \dots \geq n_k)$, 称为 n 的一个划分

例如 4 的所有划分为 4, 3 + 1, 2 + 2, 2 + 1 + 1, 1 + 1 + 1 + 1 共 5 种

1. 求 6 的所有划分
2. 求 n 的划分种类数, 要求时间复杂度 $O(n^2)$

1. 6, 5 + 1, 4 + 2, 4 + 1 + 1, 3 + 3, 3 + 2 + 1, 3 + 1 + 1 + 1, 2 + 2 + 2, 2 + 2 + 1 + 1, 2 + 1 + 1 + 1 + 1, 1 + 1 + 1 + 1 + 1 + 1 共 11 种
2. 动态规划: $q(n, m)$ 表示 n 的所有划分中最大数不超过 m 的种类数
 - 初值:
 - $q(n, m) = q(n, n) (n < m)$, 因为 n 的划分中最大数一定不超过 n , 也一定不超过 m
 - $\forall n \in \mathbb{N}_+, q(n, 1) = 1$, 因为只有 n 个 1 相加 1 种划分
 - 迭代: $q(n, m) = q(n, m - 1) + q(n - m, m)$
 - $q(n, m - 1)$ 表示最大数不超过 $m - 1$ 的划分种类数, 此时所有划分一定不含 m
 - $q(n - m, m)$ 表示至少含有 1 个 m 的划分种类数, n 中去除 m 后剩余 $n - m$, 而对 $n - m$ 的划分既可以包含 m 也可以不包含 m , 但最大数不能超过 m
 - “划分中只有 $< m$ 的数” 和 “划分中包含至少 1 个 m , 且其余部分的划分中最大数不超过 m ” 2 种情况并集的规模即为 $q(n, m)$
 - 问题解: $q(n, n)$
 - 时间复杂度: $q[n][n]$ 数组的每个元素均可在 $O(1)$ 时间内计算得到, 因此总复杂度 $O(n^2)$

6.2 Week 13–20250515

Problem 1 区间套问题: 给定若干区间, 求这些区间中包含于其他区间的区间

例如 $(1, 9), (2, 8), (3, 10), (4, 7), (5, 9)$ 的解为 $(2, 8), (4, 7), (5, 9)$

要求时间复杂度 $O(n \log n)$ ，要求使用归约求解

定义**极点**为: $\forall j \neq i, (x_j > x_i) \vee (y_j < y_i)$ ，则可根据极大点问题设计算法求出所有极点，而本问题解为所有**非极点**

Problem 2 证明：若存在时间复杂度为 $O(T(n))$ 的 $n \times n$ 下三角矩阵乘法算法，则存在时间复杂度为 $O(T(n) + n^2)$ 的 $n \times n$ 任意矩阵乘法算法

可根据下式推导复杂度关系

$$\begin{pmatrix} O & O & O \\ A & O & O \\ A & A & O \end{pmatrix} \times \begin{pmatrix} O & O & O \\ B & O & O \\ B & B & O \end{pmatrix} = \begin{pmatrix} O & O & O \\ O & O & O \\ AB & O & O \end{pmatrix}$$

Problem 3 证明：*Graham* 算法（时间复杂度 $O(n \log n)$ ）是凸包问题的最优算法

将排序问题归约为凸包问题

对于排序问题中的元素 x_i ，构造点 (x_i, x_i^2)

若 $o(n \log n)$ 时间内可以求解凸包问题，则 $o(n \log n) + O(n) = o(n \log n)$ 时间内可以求解排序问题

而排序问题的时间复杂度 $\Omega(n \log n)$ ，因此凸包问题的时间复杂度 $\Omega(n \log n)$

Problem 4 证明：基于比较构建二叉搜索树需要 $\Omega(n \log n)$ 时间

二叉搜索树的中序遍历结果为有序序列，时间复杂度 $\Theta(n)$

若 $o(n \log n)$ 时间内可以构建二叉搜索树，则 $o(n \log n) + O(n) = o(n \log n)$ 时间内可以求解排序问题

而排序问题的时间复杂度 $\Omega(n \log n)$ ，因此构建二叉搜索树的时间复杂度 $\Omega(n \log n)$

7 Homework 7

7.1 Week 14-20250522

Problem 1 证明最小公倍数问题属于P问题

辗转相除法的时间复杂度为 $O(\log \min\{m, n\})$ 是线性时间算法（问题输入规模 $O(\log m + \log n)$ 与算法时间复杂度同阶）

Problem 2 已知P和Q都是NP完全问题，P能否多项式归约到Q？

由NP完全问题的定义可知P，Q可相互多项式归约

Problem 3 是否有可能在多项式时间内判断 $G = (V, E)$ 是否存在规模为 5 的团集？

暴力遍历时间复杂度 $C \cdot \binom{n}{5} = O(n^5)$ 是多项式时间算法

7.2 Week 15–20250529

Problem 1 已知顶点覆盖问题是NP完全问题；若图为欧拉图，能否设计出多项式时间的确定性算法？

不能，可通过证明该问题为NP完全问题说明

- NP性：易知多项式时间内可验证任意 k 个点是否为 G 的顶点覆盖
- 归约：一般图 $G = (V, E)$ 的顶点覆盖 $\rightarrow$ 欧拉图 $G' = (V', E')$ 的顶点覆盖
 其中 $V' = V + \{x, y, z\}$, $E' = E + \{(x, y), (x, z), (y, z), (x, odd) | odd \in V \wedge d(odd) = 2k + 1, k \in \mathbb{N}\}$ （即向 G 中添加 3 个顶点 x, y, z ，将 x, y, z 两两连接，并将 x 与 G 中所有度数为奇数的点相连接，构成 G' ；由于 G 中度数为奇数的点一定为偶数个，所以 x, y, z 的度数均为偶数，同时消去了 G 中原有的奇点，保证 G' 为欧拉图）
 - 若 G 找到了一个 $k - 2$ 个顶点的顶点覆盖 R ，则 $R + x + y$ 或 $R + x + z$ 为 G' 的顶点覆盖
 - 若 G' 找到了一个 k 个顶点的顶点覆盖 R' ，则 R' 必然包含 x, y, z 中恰好 2 个顶点（因为只包含 1 个顶点不能覆盖到 $E' - E$ 的所有边；而包含全部 3 个顶点的顶点覆盖又可去掉 y 或 z ，剩余部分仍为顶点覆盖，与最小性矛盾），且一定包含 x （否则，若 $\{y, z\} \subset R'$ ，则用 x 替换 y, z 之一仍为顶点覆盖）
 此时 G' 删去 x, y, z 及相关联的边，剩余部分的一个顶点覆盖为 R' 删去 x, y, z 中的顶点，其规模为 $k - 2$
- 最后，由于归约过程在多项式时间内可完成，所以由“一般图的顶点覆盖问题为NP完全问题”可推出“欧拉图的顶点覆盖问题为NP完全问题”

Problem 2 证明3-MAX-SAT问题为NP完全问题

3-MAX-SAT问题是最优化问题，对应的判定问题3-SAT问题是NP完全问题，二者NP完全性相同，因此3-MAX-SAT问题也为NP完全问题

Problem 3 已知哈密顿回路问题是NP完全问题，证明旅行商问题（将城市与城市之间的路抽象为无向加权完全图，求一条遍历所有城市且最短的路径）也为NP完全问题

问题“求图 $G = (V, E)$ 的哈密顿回路”的解与问题“求 $G' = (V, E', W)$ 旅行商的最短路径”的解相同（其中 E' 为使 G 成为完全图的边集， W 为各边权值：在 E 中的边权值为 1，不在 E 中的边权值为 ∞ ）

而问题可以相互多项式归约，因此旅行商问题也为NP完全问题